VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018.



A Cryptography and Network Security Project Report on

"Project Citadel-A Enhancement of Hill Cipher"

Submitted in the partial fulfillment of the requirement for the award of degree of Bachelor of Engineering

In

Computer Science and Engineering

By

LIKHITH U **(1OX22CS088)**

MANTHAN S **(1OX22CS099)**

Under the guidance of

Prof. Jesy Janet Kumari,
Assistant Professor



Department of Computer Science and Engineering

THE OXFORD COLLEGE OF ENGINEERING

Bommanahalli, Bangalore 560068

2025-2026

THE OXFORD COLLEGE OF ENGINEERING

Hosur Road, Bommanahalli, Bengaluru-560068

(Affiliated to VISVESVARAYA TECHNOLOGICAL UNIVERSITY, Belagavi)

Department of Computer Science and Engineering

CERTIFICATE

Certified that the Project work entitled **"Project Citadel-A Enhancement of Hill Cipher"** carried out by Likhith U (1OX22CS088) and Manthan S (1OX22CS099), Bonafide students of **The Oxford College of Engineering, Bengaluru** in partial fulfillment for the award of Degree of Bachelor of Engineering in Computer Science and Engineering of the **Visvesvaraya Technological University**, Belagavi, during the year **2025-2026**. It is certified that all corrections/suggestions indicated for the assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Prof. Jesy Janet Kumari Assistant Professor, TOCE	Dr. Kanagavalli R Prof. & Head Dept of CSE, TOCE	Dr. H N Ramesh Principal, TOCE

THE OXFORD COLLEGE OF ENGINEERING

Hosur Road, Bommanahalli, Bengaluru-560068

(Affiliated to VISVESVARAYA TECHNOLOGICAL UNIVERSITY, Belagavi)

Department of Computer Science and Engineering

DECLARATION

we, **Likhith U(1OX22CS088) and Manthan S(1OX22CS099),** the students of 6th Semester B.E, at the Department of Computer Science and Engineering, **The Oxford College of Engineering, Bengaluru** declare that the project work entitled **"Project Citadel-A Enhancement of Hill Cipher"** has been carried out by us and submitted in partial fulfillment of the course requirements for the award of degree in Bachelor of Engineering in Computer Science and Engineering discipline of **Visvesvaraya Technological University, Belagavi** during the academic year **2025- 2026.** Further, the matter embodied in the report has not been submitted previously by anybody for the award of any degree or diploma to any other university.

Name	**USN**	**Signature**
Likhith U	(1OX22CS088)	
Manthan S	(1OX22CS099)	

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned my effort with success. I consider myself proud to be a part of The Oxford family, the institution that stood by me in all my endeavors.

I have great pleasure in expressing my deep sense of gratitude to the founder chairman, **Late Sri S. Narasa Raju**, and to our chairman, **Dr. S. N. V. L. Narasimha Raju**, for providing me with excellent infrastructure and well-furnished laboratories. I would like to express my gratitude to **Dr. H. N. Ramesh**, Principal, The Oxford College of Engineering, for providing me with a congenial environment and surroundings to work in.

My heartfelt thanks to **Dr. Kanagavalli R**, Professor and Head, Department of Computer Science and Engineering, The Oxford College of Engineering, for her encouragement and support. Guidance and deadlines play a very important role in the successful completion of any project. I convey my sincere gratitude to **Prof. Jesy Janet Kumari**, Assistant Professor, Department of Computer Science and Engineering, for constantly monitoring the progress of my Project Report and setting clear deadlines. I also thank her for her immense support, guidance, specifications, and ideas, without which this Project Report would have been incomplete.

Finally, I extend my heartfelt thanks to the Department of Computer Science and Engineering, including both teaching and non-teaching staff, for their cooperation and support.

Likhith U (1OX22CS088)

Manthan S (1OX22CS099)

ABSTRACT

The Hill Cipher, first introduced by Lester S. Hill in 1929, was a foundational polyalphabetic cipher notable for its innovative use of matrix algebra in cryptography. However, despite its algebraic sophistication, the classical Hill Cipher is considered insecure by modern standards. Its primary vulnerability lies in its **inherent linearity**, a property that makes it highly susceptible to known-plaintext attacks, allowing attackers to deduce the secret key matrix with relative ease. Furthermore, its traditional operation in modulo 26 restricts its application to alphabetic text (A-Z), rendering it incapable of encrypting binary files. It also lacks a diffusion mechanism or chaining mode, resulting in **repeating patterns** in the ciphertext when identical plaintext blocks are processed[3].

This project, "Project Citadel," addresses these critical deficiencies by designing and implementing a significantly enhanced and modernized version of the Hill Cipher. Our objective is to reinforce the classical algorithm's mathematical structure by integrating two key cryptographic principles to bolster its security. First, we introduce a **non-linear substitution layer** via a custom **S-Box** (Substitution Box). This component, defined by the function $S(x) = (7x + 3) 26 for text operations, is designed to break the cipher's fatal linearity and add essential diffusion, thus mitigating its vulnerability to linear cryptanalysis.

Second, we implement **Cipher Block Chaining (CBC) mode**. In this mode, each plaintext block is XORed with the previous ciphertext block before encryption (using a random **Initialization Vector (IV)** for the first block). This chaining mechanism introduces randomness and ensures that even identical plaintext blocks produce unique ciphertext blocks, effectively obscuring patterns in the data.
To move beyond simple text, "Project Citadel" extends the cipher's functionality to support modern file encryption. This is achieved by expanding the underlying mathematical operations from modulo 26 to **modulo 256**, allowing the algorithm to process raw binary data. This enhancement enables the encryption and decryption of various file formats, including **PDFs and images**.

The project culminates in the development of a clean, interactive **web application** that serves as a practical tool for secure file encryption. The application facilitates secure key and IV management and provides a **step-by-step visualizer** for educational purposes. By successfully integrating non-linear components and a secure chaining mode, Project Citadel transforms the classical Hill Cipher from a vulnerable academic curiosity into a robust, modernized algorithm capable of providing practical security for a variety of digital files.

CONTENTS

SL NO.	CONTENTS	PAGE NO.
1	Introduction	7
2	Problem Statement	8
3	Objectives	9
4	System Architecture	10-12
5	Implementation	13-14
6	GitHub Repository	15
7	Results	16-19
8	Conclusion	20
9	Future Enhancements	21
10	References	22

1. INTRODUCTION

In the field of classical cryptography, the Hill Cipher stands out for its innovative use of linear algebra, employing matrix multiplication for polyalphabetic substitution. While historically significant, the classical Hill Cipher is fundamentally insecure by modern standards. Its primary weakness is its **inherent linearity**, which makes it exceptionally vulnerable to known-plaintext attacks. An attacker with a sufficient number of plaintext-ciphertext pairs can use algebraic methods to solve for the secret key matrix. Furthermore, the original cipher lacks a chaining mechanism, causing identical plaintext blocks to produce identical ciphertext blocks, thus leaving **repeating patterns** visible. Its traditional operation in modulo 26 also limits its application strictly to alphabetic text (A-Z), rendering it completely unsuitable for encrypting modern binary files.

This project, **"Project Citadel,"** aims to address these critical vulnerabilities by designing and implementing a modernized, secure enhancement of the Hill Cipher. The goal is to strengthen the core algorithm by integrating modern cryptographic principles to defend against common attacks and extend its utility to contemporary data formats.

To achieve this, our proposed solution introduces two key features. First, to break the cipher's linearity, we integrate a **non-linear S-Box (Substitution Box)** transformation layer. Second, to obscure data patterns and add randomness, we implement **Cipher Block Chaining (CBC) mode**. This mode ensures that the encryption of each block is dependent on the previous block, using an Initialization Vector (IV) for the first block.

Furthermore, "Project Citadel" expands the cipher's domain from modulo 26 to **modulo 256**, enabling it to process raw binary data. This critical extension allows the algorithm to encrypt and decrypt a wide range of file types, including **PDFs and images**.

The project's objectives are to analyze the weaknesses of the classical Hill Cipher , strengthen its security by integrating the S-Box and CBC mode, and develop a practical implementation. This implementation takes the form of an **interactive web application** that not only performs secure file encryption but also includes a **visualizer tool** to help users understand the step-by-step encryption and decryption process. This report details the analysis, design, and implementation of this enhanced cipher.

2. PROBLEM STATEMENT

The classical Hill Cipher is a polyalphabetic substitution cipher based on **matrix multiplication modulo 26**. While innovative for its time, its reliance on simple linear algebra results in significant security flaws and practical limitations.

The core challenges of the classical Hill Cipher are:

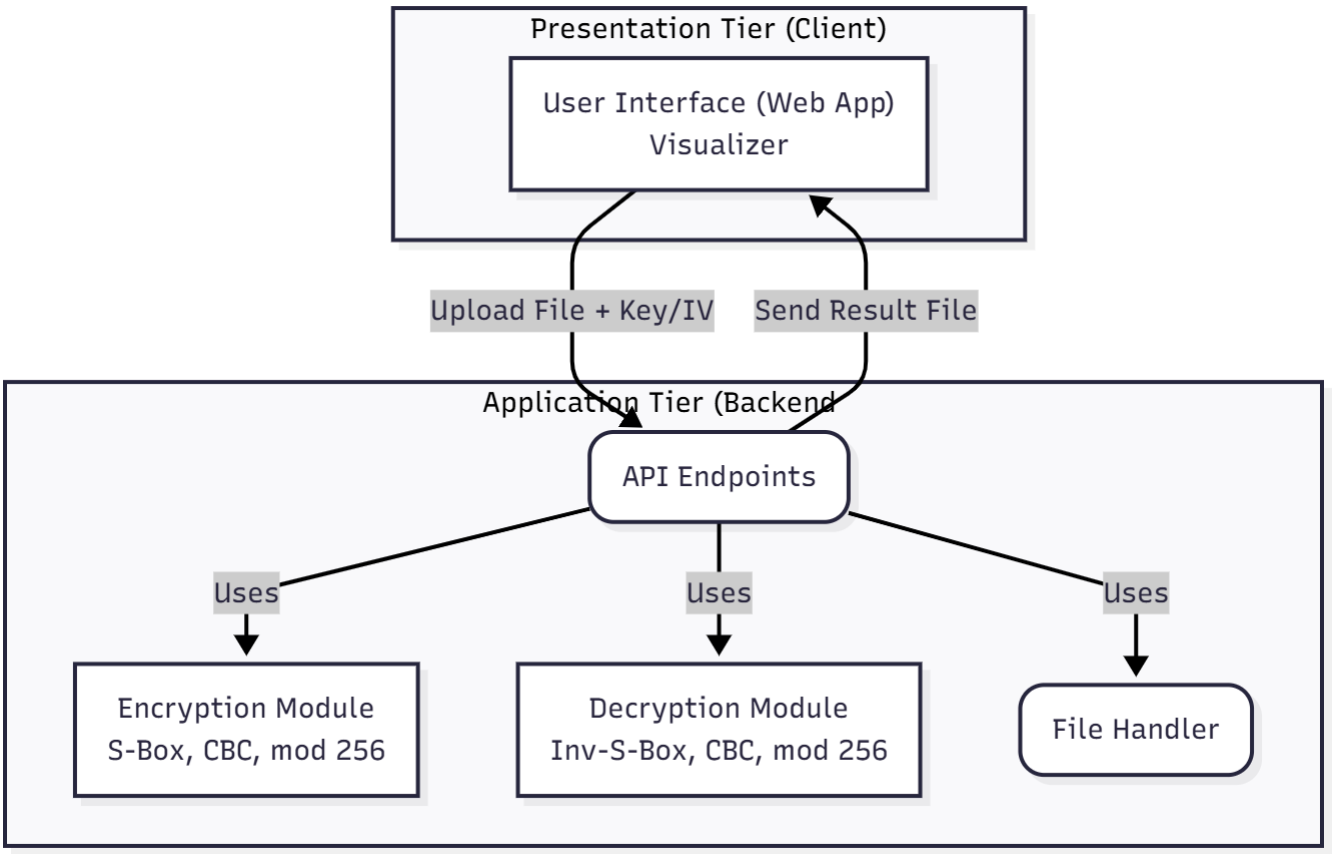
- **Linearity:** The cipher is entirely linear. This makes it highly **vulnerable to known-plaintext attacks**, where an attacker can determine the secret key matrix by solving a system of linear equations.
- **Lack of Non-Linearity:** The algorithm contains no non-linear transformation. This makes its output predictable and **breakable with standard algebraic methods**.
- **Pattern Repetition:** The cipher lacks a chaining mode. As a result, identical plaintext blocks encrypt to identical ciphertext blocks, leaving **repeating patterns** that can be exploited in cryptanalysis.
- **Limited Scope:** Its operation in modulo 26 restricts its use to the 26 letters of the alphabet (A-Z), making it **unsuitable for encrypting modern binary files** like PDFs, images, or other data formats.
- These vulnerabilities combined mean the classical Hill Cipher is **not practical for modern file security**.

3. OBJECTIVES

The primary goal of "Project Citadel" is to transform the classical Hill Cipher from a an insecure, text-only algorithm into a robust and practical tool for modern file encryption. The specific objectives to achieve this goal are as follows:

1. **Analyze the Mathematical Structure and Weaknesses of the Original Hill Cipher:** This involves a thorough review of the classical algorithm to identify and demonstrate its critical flaws. The analysis will focus on its inherent linearity, which makes it vulnerable to known-plaintext attacks, and its lack of diffusion, which leads to repeating patterns in the ciphertext.
2. **Design and Implement a New Algorithm Incorporating Non-Linear Components:** To address the core linearity weakness, a new algorithm will be designed. This objective will be met by integrating a non-linear S-Box (Substitution Box) into the encryption process. This component is intended to break the simple algebraic relationship between the plaintext, key, and ciphertext, thus strengthening the cipher against linear cryptanalysis.
3. **Integrate Cipher Block Chaining (CBC) Mode**: To eliminate repeating patterns and add randomness, the project will implement CBC mode. This involves using an Initialization Vector (IV) and a chaining mechanism, ensuring that the encryption of each data block is dependent on the previous block, thereby significantly enhancing data security.
4. **Extend Cipher Support to Binary Files (PDFs, Images):** This objective is to overcome the "mod 26" limitation of the original cipher. By re-engineering the algorithm to operate in modulo 256, the enhanced cipher will be capable of processing raw binary data, making it possible to encrypt and decrypt various file formats, including PDFs and images.
5. **Develop and Deploy a Functional Web Application for Practical Use:** A clean, modern, and interactive web interface will be built to serve as a practical implementation of the enhanced cipher. This application will allow users to upload, encrypt, and decrypt files, as well as manage the necessary secret keys and IV values securely.
6. **Provide a Visualizer Tool for Learning and Demonstration:** As an educational component, the project will include a step-by-step visualizer. This tool will allow users to input text and observe the entire encryption and decryption process, including the matrix multiplication, S-Box substitution, and CBC operations, to better understand how the algorithm works.

4. SYSTEM ARCHITECTURE



4.1. Encryption Process Flow

1. **Input:** The user accesses the Presentation Tier (web interface) and selects a file (PDF, image, or text) to encrypt. The user also provides a secret Key Matrix and a public Initialization Vector (IV).
2. **Transmission:** The file, key, and IV are sent to the Application Tier (backend server).
3. **Processing (Backend): The server's Encryption Module executes the following steps:**
    - The file is read as binary data (mod 256) and split into blocks.
    - CBC Mode (Block 1): The first plaintext block is XORed with the IV.
    - Hill Cipher: The result is multiplied by the Key Matrix (mod 256).
    - S-Box: The resulting block is passed through the non-linear S-Box for substitution. This final output is Ciphertext Block 1.
    - CBC Mode (Block 2+): The second plaintext block is XORed with Ciphertext Block 1 (the chaining mechanism).
    - This new result is processed by the Hill Cipher and S-Box to create Ciphertext Block 2. This process repeats for all blocks.
4. **Output:** All processed ciphertext blocks are assembled into a single file, encrypted.enc.
5. **Response:** The server sends the encrypted.enc file back to the user, who can then download it.

4.2. Decryption Process Flow

1. **Input:** The user uploads the encrypted.enc file, along with the *exact* **Key Matrix** and **IV** that were used to encrypt it.
2. **Transmission:** The encrypted file, key, and IV are sent to the **Application Tier**.
3. **Validation:** The server's **Decryption Module** verifies it has the necessary components. Decryption fails if the key or IV is incorrect.
4. **Processing (Backend):** The server executes the *reverse* process:
    - The encrypted file is split into ciphertext blocks.
    - **Decryption (Block 1): Ciphertext Block 1** is processed.
    - **Inverse S-Box:** The block is passed through the **Inverse S-Box** (reverse substitution).
    - **Inverse Hill:** The result is multiplied by the **Inverse Key Matrix**.
    - **CBC Reverse:** This result is **XORed** with the **IV** to recover **Plaintext Block 1**.
    - **Decryption (Block 2+): Ciphertext Block 2** is processed by the **Inverse S-Box** and **Inverse Hill** matrix.
    - **CBC Reverse:** The result is **XORed** with **Ciphertext Block 1** (the previous ciphertext block) to recover **Plaintext Block 2**. This repeats for all blocks.

5. **Output:** All decrypted plaintext blocks are reassembled into the original file (e.g., original.pdf).
6. **Response:** The server sends the original, decrypted file back to the user for download.

4.3. Technologies Used

The project's architecture is split into a frontend (client-side) and a backend (server-side), each built with a specific set of modern technologies.

1. Frontend (Presentation Tier)

- **React.js (with JSX):** The core JavaScript library used to build the "modern web interface". It manages the application's state and component-based structure, handling all user interactions, file uploads, and the logic for the "visualizer tool for learning".
- **Vite:** The high-performance build tool and development server used to bundle the React application.
- **HTML & CSS:** The foundational technologies used to structure the web pages and apply the "clean, modern UI with bright styling".
- **JSON:** Used as the data format for configuration files and for exchanging data between the React frontend and the Python backend API.

2. Backend (Application Tier)

- **Python:** The primary backend language used to implement the core cryptographic logic. This includes all matrix multiplication, S-Box transformations, CBC chaining, and binary file handling (mod 256).
- **REST API:** The architectural style for communication, allowing the frontend to send plaintext/ciphertext data and receive processed files from the backend.

3. Cryptographic Core

- **Modular Arithmetic:** The mathematical foundation, operating in **mod 256** for binary file data and **mod 26** for the text-based visualizer.
- **Custom S-Box:** A non-linear substitution function, defined for text as .
- **Cipher Block Chaining (CBC):** The cryptographic mode of operation used to add randomness and hide patterns, requiring an **Initialization Vector (IV)**.

5. IMPLEMENTATION

Phase 1: Research and Algorithm Design

The initial phase involved analyzing the classical Hill Cipher to identify its vulnerabilities, specifically its linearity and lack of diffusion. The design for the enhanced algorithm was then established:

- **Non-Linearity:** A custom **S-Box** was designed to be applied after the matrix multiplication. For the text-based visualizer, this was $S(x) = (7x + 3) \bmod 26$.
- **Chaining: CBC (Cipher Block Chaining) mode** was selected to hide plaintext patterns by using an **IV** and chaining blocks.
- **File Support:** The algorithm's mathematics were extended to operate in **mod 256** to support binary data from files like PDFs and images.
- **Decryption:** The inverse operations, including the Inverse S-Box and the calculation of the inverse key matrix, were mathematically defined to enable decryption.

Phase 2: Backend Development (Python)

The core cryptographic engine was built first using Python.

- **Core Crypto Functions:** We implemented the foundational functions for matrix multiplication, finding the modular inverse of the key matrix, and the S-Box/Inverse S-Box transformations. These functions were built to operate in both mod 26 (for the visualizer) and mod 256 (for file encryption).
- **File Handling:** Python was used to read binary data from uploaded files (PDFs, images) and to write the processed binary data into the resulting .enc file.
- **CBC Mode Implementation:** The CBC logic was wrapped around the core functions. This involved managing the IV for the first block and using the output of the previous ciphertext block as the vector for the next block's XOR operation

Phase 3: API Development

Using a Python web framework (like Flask or Django), a **REST API** was created to serve as the bridge between the frontend and the backend logic.

- **/encrypt Endpoint:** An endpoint was created to receive a file, Key Matrix, and IV from the frontend. It passes these inputs to the Python encryption script and returns the resulting encrypted.enc file.
- **/decrypt Endpoint:** A similar endpoint was created to receive an .enc file, Key Matrix, and IV. It validates these inputs and, if correct, runs the decryption script, returning the original file.

- **visualize Endpoint:** A separate endpoint was developed to handle text-only requests from the visualizer tool, returning a JSON object with the step-by-step transformations.

Phase 4: Frontend Development (React)

The "modern web interface" was built using **React.js**.

- **Project Setup:** The project was initialized using **Vite** for a fast development environment.
- **Component Creation (JSX):** We developed several **React components** to build the UI:
    - A file-handling component for uploading PDFs, images, and text files.
    - Input components for the Key Matrix and IV values.
    - A "Visualizer" component to display the step-by-step encryption process for text.
- **Styling (CSS):** CSS was used to implement the "clean, modern UI with bright styling".

Phase 5: Integration and Testing

The final phase involved connecting the React frontend to the Python backend.

- **API Calls:** JavaScript's fetch (or a similar library) was used in the React components to send POST requests (containing the files and **JSON** data) to the Python API endpoints.
- **End-to-End Testing:** We performed extensive testing by encrypting various file types (PDF, images), downloading the .enc file, and then uploading that file with the correct key and IV to ensure a perfect, bit-for-bit decryption and recovery of the original file.

Challenges Faced

During development, our team (Manthan S and Likhith U) overcame three main technical challenges:

- **Complex Cryptographic Math:** Extending the cipher from mod 26 to mod 256 for files was non-trivial. We had to implement a check to ensure the key matrix's determinant was odd (coprime to 256) to be invertible. We also had to design a new, non-linear, and perfectly reversible **S-Box** for all 256-byte values.
- **Data Integrity and Padding:** The CBC mode requires fixed-size blocks. We had to implement a **padding scheme** (like PKCS#7) to handle files whose size wasn't a perfect multiple of the block size. Ensuring this padding was added and removed perfectly was critical, as a single bit error would corrupt an entire PDF or image.
- **Binary File Transmission:** A significant challenge was sending raw binary files from the **React (JSX) frontend** to the **Python backend** over an API. We had to ensure the file data was transmitted and reconstructed as a byte stream without any encoding errors or corruption.

6. GITHUB REPOSITORY

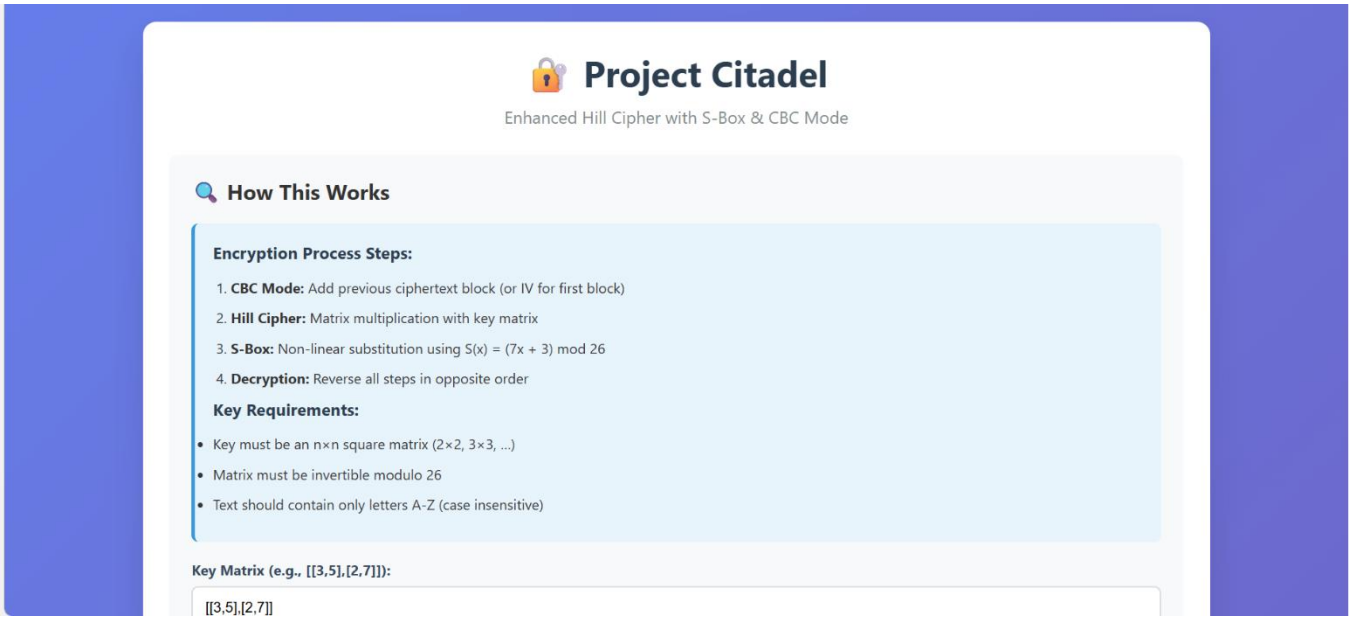
The entire implementation of the project “Project Citadel-A Enhancement of Hill Cipher” can be accessed at the following GitHub repository:

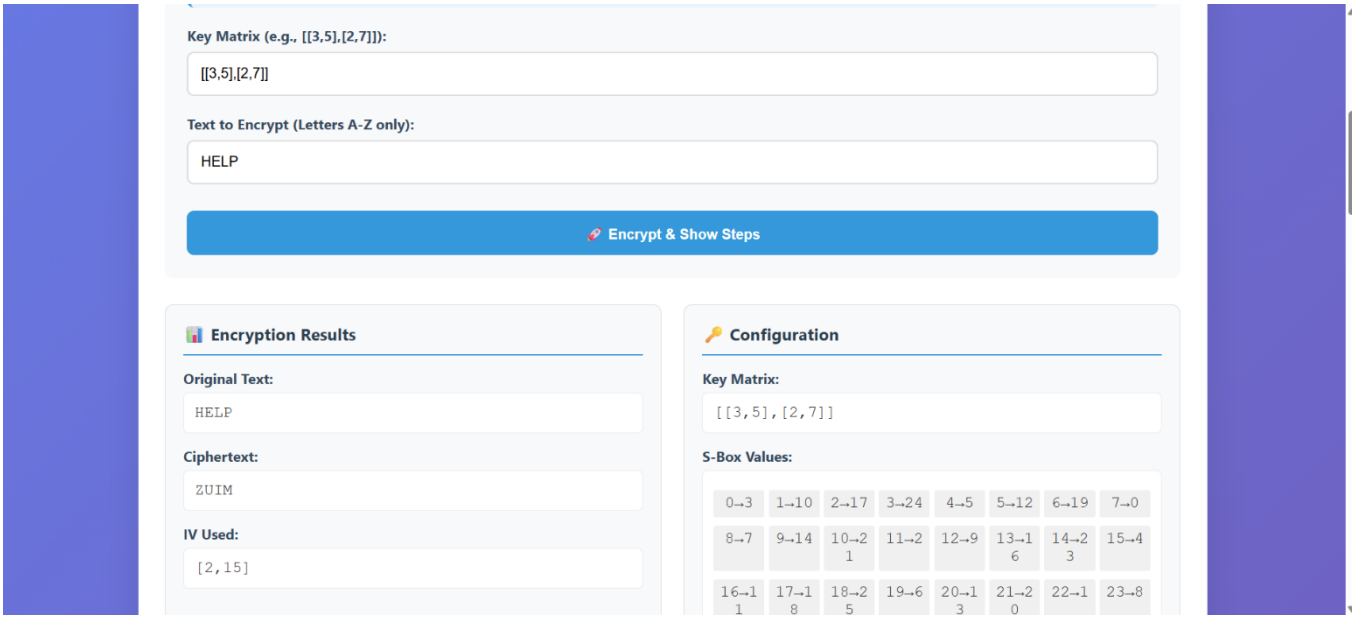
manthans2004/Project-Citadel: A modernized enhancement of the Hill Cipher incorporating non-linear diffusion and CBC mode.

This repository contains:

- The main web application for file encryption (in the citadel-app branch).
- The separate visualizer tool for learning (in the visualization-project-app branch).

7.RESULTS


Project Citadel
Enhanced Hill Cipher with S-Box & CBC Mode
How This Works
Encryption Process Steps:
1. CBC Mode: Add previous ciphertext block (or IV for first block)
2. Hill Cipher: Matrix multiplication with key matrix
3. S-Box: Non-linear substitution using S(x) = (7x + 3) mod 26
4. Decryption: Reverse all steps in opposite order
Key Requirements:
Key must be an n×n square matrix (2×2, 3×3, ...)
Matrix must be invertible modulo 26
Text should contain only letters A-Z (case insensitive)
Key Matrix (e.g., [[3,5],[2,7]]):
[[3,5],[2,7]]


Key Matrix (e.g., [[3,5],[2,7]]):
[[3,5],[2,7]]
Text to Encrypt (Letters A-Z only):
HELP
Encrypt & Show Steps
Encryption Results
Original Text:
HELP
Ciphertext:
ZUIM
IV Used:
[2,15]
Configuration
Key Matrix:
[[3,5],[2,7]]
S-Box Values:
0→3 1→10 2→17 3→24 4→5 5→12 6→19 7→0
8→7 9→14 10→21 11→2 12→9 13→16 14→23 15→4
16→11 17→18 18→25 19→6 20→13 21→20 22→1 23→8

Step-by-Step Encryption Process

Initial Setup

Plaintext: "HELP"
Key Matrix: [[3, 5], [2, 7]]
IV: [2, 15]
Blocks: [[7, 4], [11, 15]]

Block 1 Encryption

CBC: [7, 4] + [2, 15] = [9, 19]
Hill: [[3, 5], [2, 7]] × [9, 19] = [18, 21]
S-Box: [18, 21] → [25, 20]

Cipher Block: [25, 20]

Step-by-Step Decryption Process

Initial Setup

Ciphertext: "ZUIM"
IV: [2, 15]
Inverse Key: [[3, 9], [14, 5]]
Blocks: [[25, 20], [8, 12]]

Block 1 Decryption

Inverse S-Box: [25, 20] → [18, 21]
Inverse Hill: [[3, 9], [14, 5]] × [18, 21] = [9, 19]
CBC: [9, 19] - [2, 15] = [7, 4]

Plain Block: [7, 4]

Block 2 Decryption

Inverse S-Box: [8, 12] → [23, 5]
Inverse Hill: [[3, 9], [14, 5]] × [23, 5] = [10, 9]
CBC: [10, 9] - [25, 20] = [11, 15]

Plain Block: [11, 15]

Final Result

Decrypted: "HELP"

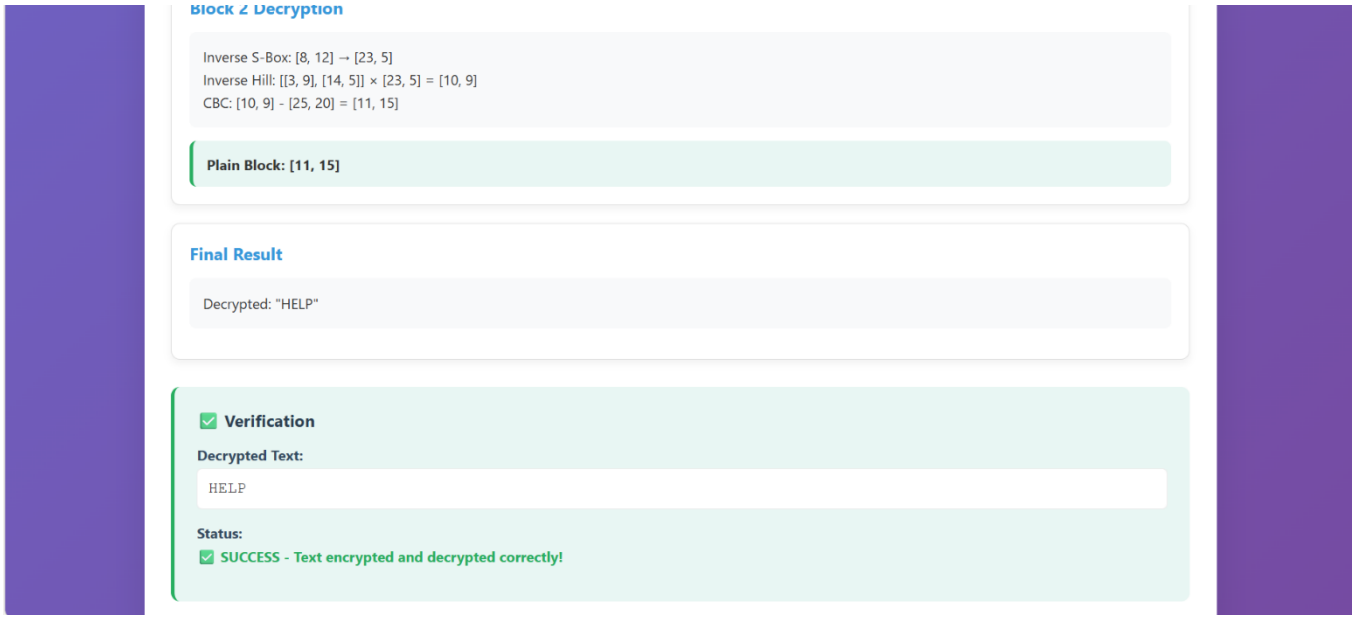
✅ **Verification**

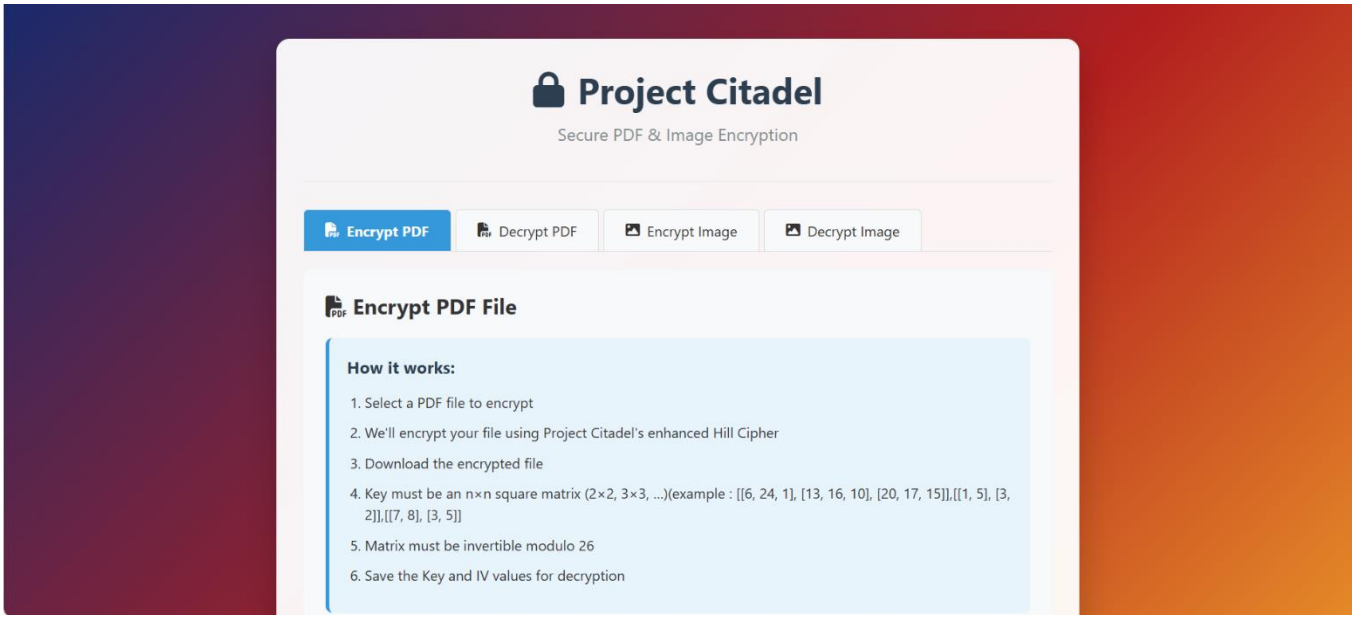
Decrypted Text:

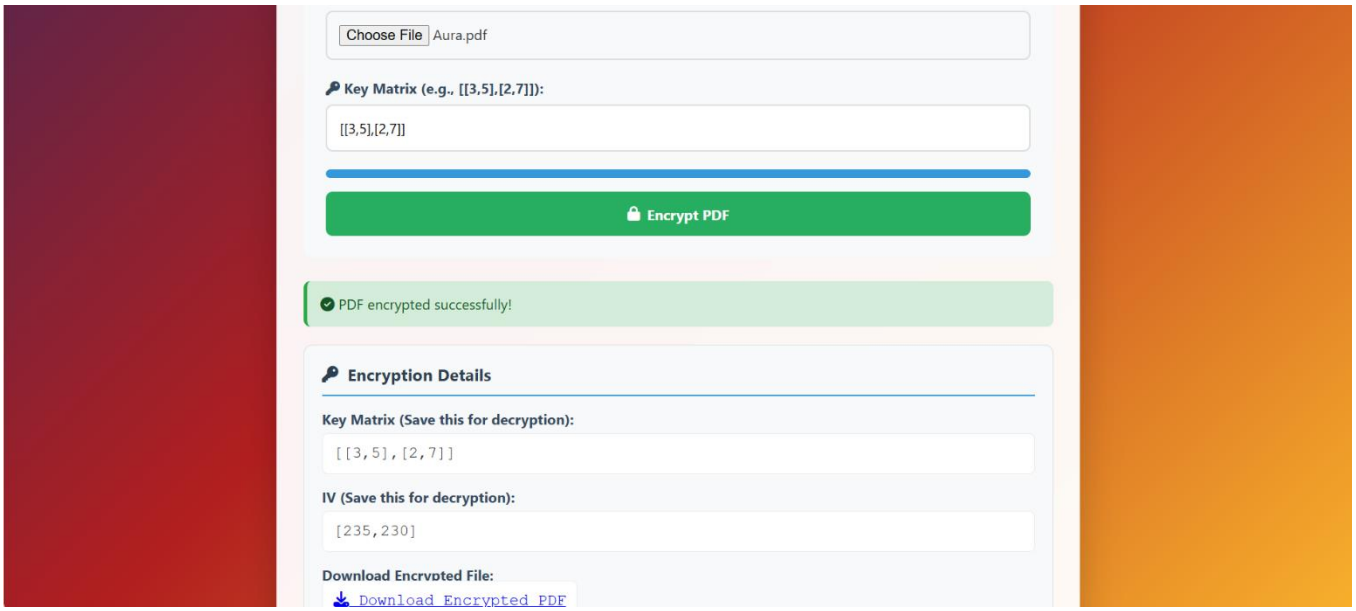
HELP

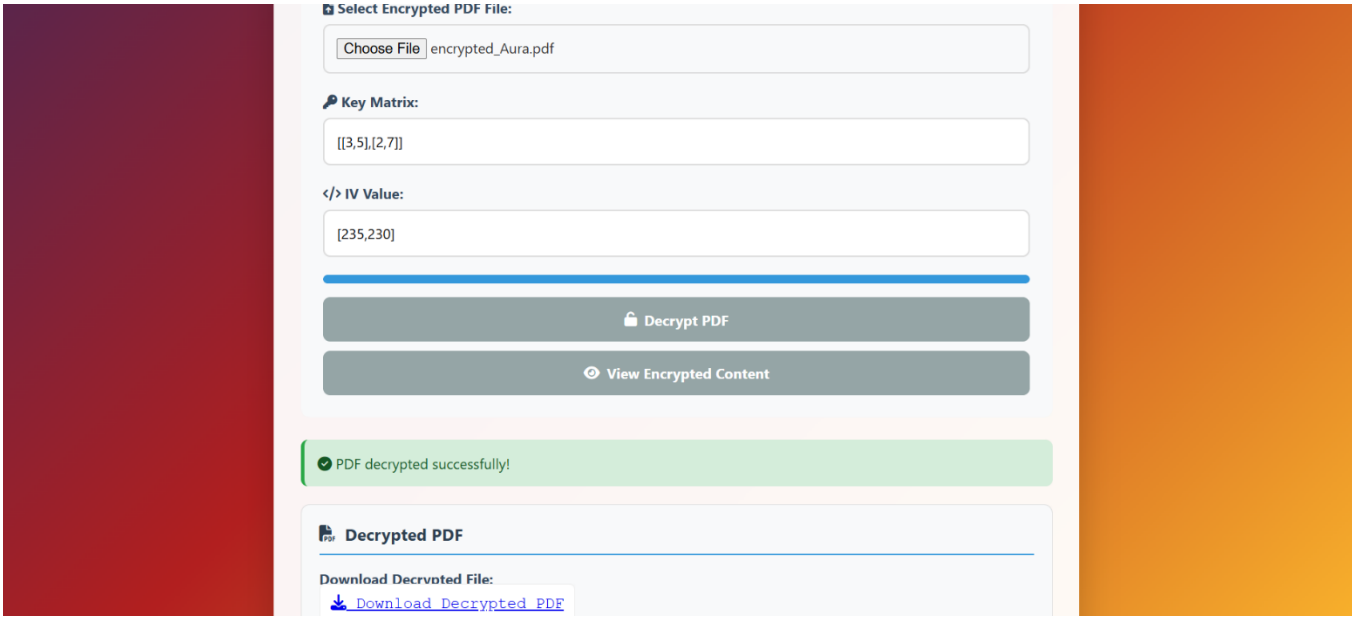
Status:

✅ SUCCESS - Text encrypted and decrypted correctly!


Block 2 Decryption
Inverse S-Box: [8, 12] → [23, 5]
Inverse Hill: [[3, 9], [14, 5]] × [23, 5] = [10, 9]
CBC: [10, 9] - [25, 20] = [11, 15]
Plain Block: [11, 15]
Final Result
Decrypted: "HELP"
Verification
Decrypted Text:
HELP
Status:
SUCCESS - Text encrypted and decrypted correctly!


Project Citadel
Secure PDF & Image Encryption
Encrypt PDF
Decrypt PDF
Encrypt Image
Decrypt Image
Encrypt PDF File
How it works:
1. Select a PDF file to encrypt
2. We'll encrypt your file using Project Citadel's enhanced Hill Cipher
3. Download the encrypted file
4. Key must be an n×n square matrix (2×2, 3×3, ...)(example : [[6, 24, 1], [13, 16, 10], [20, 17, 15]],[[1, 5], [3, 2]],[[7, 8], [3, 5]]
5. Matrix must be invertible modulo 26
6. Save the Key and IV values for decryption


Choose File Aura.pdf
Key Matrix (e.g., [[3,5],[2,7]]):
[[3,5],[2,7]]
Encrypt PDF
PDF encrypted successfully!
Encryption Details
Key Matrix (Save this for decryption):
[[3,5],[2,7]]
IV (Save this for decryption):
[235,230]
Download Encrypted File:
Download Encrypted PDF


Select Encrypted PDF File:
Choose File encrypted_Aura.pdf
Key Matrix:
[[3,5],[2,7]]
</> IV Value:
[235,230]
Decrypt PDF
View Encrypted Content
PDF decrypted successfully!
Decrypted PDF
Download Decrypted File:
Download Decrypted PDF

8.CONCLUSION

The classical Hill Cipher, while historically significant for its use of matrix algebra, is fundamentally insecure for modern applications. Its **inherent linearity** makes it vulnerable to known-plaintext attacks, and its traditional **mod 26** operation limits it to text, rendering it incapable of securing files.

This project, **"Project Citadel,"** successfully addressed these critical deficiencies by designing and implementing a modernized, secure enhancement[3]. We have met all our objectives by:

- Breaking the cipher's linearity by integrating a **non-linear S-Box** ($S(x) = (7x + 3) mod 26.
- Obscuring plaintext patterns and adding randomness through the implementation of **Cipher Block Chaining (CBC) mode**.
- Extending the algorithm to **mod 256** to support the encryption of binary files, including **PDFs and images**.

The project has delivered a functional, full-stack **web application**, built with a React (JSX) frontend and a Python backend. This application, found in the citadel-app branch, provides a practical tool for secure file encryption and decryption. Furthermore, the visualization-project-app branch provides a valuable **visualizer tool** for educational purposes.

In conclusion, Project Citadel successfully transforms the Hill Cipher from an academic curiosity into a robust, layered, and practical algorithm, demonstrating how classical ciphers can be effectively strengthened with modern cryptographic principles.

9.FUTURE ENHANCEMENTS

The following enhancements could be implemented to improve Project Citadel's security, usability, and performance.

- **Upgrade Cryptographic Security:**
  - Replace the current S-Box with a more robust, non-linear function, such as the **Rijndael S-Box (used in AES)**, to provide stronger confusion.
  - Implement an **Authenticated Encryption (AEAD)** mode like **GCM** instead of CBC. This would provide data integrity and authenticity, protecting against tampering (which CBC does not).
- **Improve Usability with Key Derivation:**
  - Implement a **Key Derivation Function (KDF)** like **Argon2**. This would allow users to enter a single memorable password, from which the invertible Key Matrix and IV could be securely generated, removing the burden of manual key management.
- **Boost Backend Performance:**
  - Refactor the Python backend to use the **NumPy** library for all matrix multiplication and S-Box transformations. This would dramatically increase the encryption and decryption speed for large binary files.

10.REFERENCES

1. Stallings, W. (2020). *Cryptography and Network Security: Principles and Practice* (8th ed.). Pearson.

2. National Institute of Standards and Technology (NIST). (2001). *SP 800-38A: Recommendation for Block Cipher Modes of Operation*.

3. Hill, L. S. (1929). Cryptography in an Algebraic Alphabet. *The American Mathematical Monthly*, 36(6), 306□312.

4. GeeksforGeeks. (2023). *Hill Cipher*. [Online]. Available: [Insert URL you used, e.g., https://www.geeksforgeeks.org/hill-cipher]

5. Python Software Foundation. (2023). *Python 3.11.5 documentation*. [Online]. Available: https://docs.python.org/3/

6. React Team. (2023). *React Documentation*. [Online]. Available: https://react.dev/

7. Vite Team. (2023). *Vite Documentation*. [Online]. Available: https://vitejs.dev/